

Compiladores 2026–2
Práctica 3
Análisis sintáctico descendente predictivo

Prof. Ariel Adara Mercado Martínez
Ayud. Carlos Gerardo Acosta Hernández
Adud. Luis Mario Escobar Rosales

Rodrigo André Decuir Fuentes
No. Cuenta: 320022711
ojo@ciencias.unam.mx

Fecha de entrega: 28 de abril



Ejercicios

Para la gramática $G = (N, \Sigma, P, S)$ descrita por las siguientes producciones:

```
P = {
  programa → declaraciones sentencias
  declaraciones → declaraciones declaracion | declaracion
  declaracion → tipo lista-var ;
  tipo → int | float
  lista_var → lista_var , identificador | identificador
  sentencias → sentencias sentencia | sentencia
  sentencia → identificador = expresion ; | if ( expresion ) sentencias
else
  sentencias | while ( expresión ) sentencias
  expresion → expresion + expresion | expresion - expresion | expresion *
  expresion | expresion / expresión | identificador | numero
  expresion → ( expresion )
}
```

1. Determinar en un archivo Readme, en formato Markdown (.md) o LaTeX (.tex) – con su respectivo PDF, para este último –, los conjuntos N, Σ y el símbolo inicial S . (0.5 pts.)

Respuesta: $N = \{\text{programa, declaraciones, sentencias, declaracion, tipo, lista_var, sentencia, expresion}\}$, $\Sigma = \{\text{identificador, int, float, if, (,), while, *, -, +, /, numero, else, ;, ', ', =}\}$, $S = \{\text{programa}\}$.

- a. Mostrar en el archivo el proceso de eliminación de ambigüedad o justificar, en caso de no ser necesario. (0.25 pts.)

Respuesta: Es necesario eliminar la ambigüedad porque no hay precedencia de operadores ni asociatividad. Lo podemos hacer transformando la producción "expresion" del siguiente modo (agregando las producciones termino y factor):

```
expresion -> expresion + termino | expresion - termino | termino
termino -> termino * factor | termino / factor | factor
factor -> identificador | numero | (expresion)
```

- b. Mostrar en el archivo el proceso de eliminación de la recursividad izquierda o justificar, en caso de no ser necesario. (0.25 pts.)

Respuesta: si hay recursividad izquierda, por lo que hay que eliminarla:

Producciones	Reglas
declaraciones -> declaraciones declaracion declaracion declaraciones -> declaracion declaraciones' declaraciones' -> declaracion declaraciones' ϵ	$A \rightarrow A\alpha \mid \beta$ $A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \epsilon$
lista_var -> lista_var, identificador identificador lista_var -> identificador lista_var' lista_var' -> , identificador lista_var' ϵ	$A \rightarrow A\alpha \mid \beta$ $A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \epsilon$

<pre> sentencias -> sentencias sentencia sentencia sentencias -> sentencia sentencias' sentencias' -> sentencia sentencias' ε </pre>	$A \rightarrow A\alpha \mid \beta$ $A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \epsilon$
<pre> expresion -> expresion + termino expresion - termino termino expresion -> termino expresion' expresion' -> + termino expresion' - termino expresion' ε </pre>	$A \rightarrow A\alpha \mid \beta$ $A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \epsilon$
<pre> termino -> termino * factor termino / factor factor termino -> factor termino' termino' -> * factor termino' / factor termino' ε. </pre>	$A \rightarrow A\alpha \mid \beta$ $A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \epsilon$

- c. Mostrar en el archivo el proceso de factorización izquierda o justificar, en caso de no ser necesario. (0.25 pts.)

Respuesta: No es necesario, ya que no hay producciones que compartan prefijos.

- d. Mostrar en el archivo los nuevos conjuntos N' y P' que definen G' . (0.25 pts.)

Respuesta:

$N = \{\text{programa, declaraciones, declaraciones', sentencia, sentencias', declaracion, tipo, lista_var, lista_var', sentencia, expresion, expresion', termino, termino', factor}\}.$

```

P = {
programa -> declaraciones sentencias
declaraciones -> declaracion declaraciones'
declaraciones' -> declaracion declaraciones' | ε
declaracion -> tipo lista-var ;
tipo -> int | float
lista_var -> identificador lista_var'
lista_var' -> , identificador lista_var' | ε
sentencias -> sentencia sentencias'
sentencias' -> sentencia sentencias' | ε
sentencia -> identificador = expresion;
| if (expresion) sentencias else sentencias
| while (expresión) sentencias
expresion -> termino expresion'
expresion' -> + termino expresion' | - termino expresion' | ε
termino -> factor termino'
termino' -> * factor termino' | \ factor termino' | ε
factor -> identificador | numero | (expresion)
}

```

2. Mostrar en el archivo la construcción de los conjuntos FIRST de la gramática G' . (1 pt.)

Respuesta:

$\text{FIRST}(\text{factor}) = \{\text{identificador, numero, (}\}$

$\text{FIRST}(\text{termino}') = \{*, /, \epsilon\}$

$\text{FIRST}(\text{termino}) = \{\text{identificador, numero, (}\}$

$\text{FIRST}(\text{expresion}') = \{+, -, \epsilon\}$

$\text{FIRST}(\text{expresion}) = \{ \text{identificador, numero, (} \}$
 $\text{FIRST}(\text{sentencia}) = \{ \text{identificador, if, while } \}$
 $\text{FIRST}(\text{sentencias}') = \{ \text{identificador, if, while, } \epsilon \}$
 $\text{FIRST}(\text{sentencias}) = \{ \text{identificador, if, while } \}$
 $\text{FIRST}(\text{lista_var}') = \{ , \epsilon \}$
 $\text{FIRST}(\text{lista_var}) = \{ \text{identificador} \}$
 $\text{FIRST}(\text{tipo}) = \{ \text{int, float } \}$
 $\text{FIRST}(\text{declaracion}) = \{ \text{int, float } \}$
 $\text{FIRST}(\text{declaraciones}') = \{ \text{int, float, } \epsilon \}$
 $\text{FIRST}(\text{declaraciones}) = \{ \text{int, float } \}$
 $\text{FIRST}(\text{programa}) = \{ \text{int, float } \}$

3. Mostrar en el archivo la construcción de los conjuntos FOLLOW de la gramática G' . (1 pt.)

Respuesta:

$\text{FOLLOW}(\text{programa}) = \{ \$ \}$
 $\text{FOLLOW}(\text{sentencias}) = \{ \$, \text{else } \}$
 $\text{FOLLOW}(\text{sentencias}') = \{ \$, \text{else } \}$
 $\text{FOLLOW}(\text{sentencia}) = \{ \text{identificador, if, while, } \$, \text{else } \}$
 $\text{FOLLOW}(\text{declaraciones}) = \{ \text{identificador, if, while } \}$
 $\text{FOLLOW}(\text{declaraciones}') = \{ \text{identificador, if, while } \}$
 $\text{FOLLOW}(\text{declaracion}) = \{ \text{int, float, identificador, if, while } \}$
 $\text{FOLLOW}(\text{tipo}) = \{ \text{identificador } \}$
 $\text{FOLLOW}(\text{lista_var}) = \{ ; \}$
 $\text{FOLLOW}(\text{lista_var}') = \{ ; \}$
 $\text{FOLLOW}(\text{expresion}) = \{ ;,), \text{else } \}$
 $\text{FOLLOW}(\text{expresion}') = \{ ;,), \text{else } \}$
 $\text{FOLLOW}(\text{termino}) = \{ +, -, ;,), \text{else } \}$
 $\text{FOLLOW}(\text{termino}') = \{ +, -, ;,), \text{else } \}$
 $\text{FOLLOW}(\text{factor}) = \{ *, /, +, -, ;,), \text{else } \}$

4. Mostrar en el archivo la construcción de la tabla de análisis sintáctico predictivo para G' . (1 pt.)

Respuesta:

La tabla inicia en la siguiente página, dado que es muy ancha... opte por particionarla; por cuestión de espacio y visibilidad.

No Terminal	int / float	identificador	numero
programa declaraciones declaraciones' declaracion tipo lista_var lista_var' sentencias sentencias' sentencia expresion expresion' termino termino' factor	declaraciones sentencias declaracion declaraciones' declaracion declaraciones' tipo lista_var ; int float	ϵ identificador lista_var' sentencia sentencias' sentencia sentencias' identificador = expresion ; termino expresion' factor termino' identificador	 termino expresion' factor termino' numero

No Terminal	if	else	while
programa declaraciones declaraciones' declaracion tipo lista_var lista_var' sentencias sentencias' sentencia expresion expresion' termino termino' factor	ϵ sentencia sentencias' sentencia sentencias' if (expresion) sentencias else sentencias	 ϵ ϵ ϵ	ϵ sentencia sentencias' sentencia sentencias' while (expresion) sentencias

No Terminal	=	+ / -	* / /
programa declaraciones declaraciones' declaracion tipo lista_var lista_var' sentencias sentencias' sentencia expresion expresion' termino termino' factor		+ termino expresion'- termino expresion' ϵ	* factor termino'/ factor termino'

No Terminal	(	)	;	,	\$
programa declaraciones declaraciones' declaracion tipo lista_var lista_var' sentencias sentencias' sentencia expresion expresion' termino termino' factor				ϵ , identificador lista_var' termino expresion' factor termino' (expresion)	ϵ ϵ ϵ

5. Sustituir el contenido del Analizador Léxico (lexer.ll) con el implementado en la segunda práctica. (0.5 pts.)

Respuesta: Listo.

6. Definir en un comentario de Symbols.hpp la gramática G' . (0.05 pts.)

Respuesta: Listo.

7. Definir Σ en un enum de Symbols.hpp. (0.10 pts.)

Respuesta: Listo.

8. Definir N' en un enum de Symbols.hpp. (0.10 pts.)

Respuesta: Listo.

9. Cargar $N' \cup \Sigma$ en ParserLL.cpp. (0.25 pts.)

Respuesta: Listo.

10. Cargar P' en ParserLL.cpp. (0.25 pts.)

Respuesta: Listo.

11. Cargar la tabla de análisis sintáctico predictivo en ParserLL.cpp. (0.25) pts.)

Respuesta: Listo.

12. Implementar el algoritmo de análisis sintáctico de descenso predictivo en ParserLL.cpp de modo que el programa acepte el archivo prueba. (4 pts.)

Respuesta: Listo.

Extras

Documentar el código. (0.25pts) - No lo hice.

Proponer 4 archivos de prueba nuevos, 2 válidos y 2 inválidos. (0.25pts) - No lo hice.